



Blazor Essencial

Criando componentes aninhados usando
EventCallback





Criando componentes filhos e aninhados e usando EventCallback

Componente Pai

Home.razor

Componente Filho

Aviso.razor

Tarefas.json

Lista de tarefas

⚠ Lista de Tarefas			
Ação	Data	Descrição	Concluida
	11/08/2019 00:00:00	Fazer o curso de Blazor	☐ False
	05/09/2019 00:00:00	Estudar Blazor	☐ False
Tarefa	Informe a tarefa	Registrar Tarefa	

Componente Pai

Home.razor

Componente Filho

ListaTarefas.razor
EventCallback

Componente Neto

TarefaIndividual.razor
EventCallback

Tarefas.json



Refatorando para componentes filhos (ListaTarefas.razor)

Home.razor

```
@page "/"
@rendermode InteractiveServer
@Inject HttpClient http
@Inject NavigationManager navManager

<PageTitle>Home</PageTitle>
<Aviso Mensagem="Lista de Tarefas"/>

...

@code{
    List<Tarefa> tarefas = new List<Tarefa>();
    ...
}
```



Lista de tarefas

 Lista de Tarefas

Ação	Data	Descrição	Concluida
	11/08/2019 00:00:00	Fazer o curso de Blazor	☐ False
	05/09/2019 00:00:00	Estudar Blazor	☐ False

Tarefa

Carrega os dados, exibe a lista de tarefas e ainda permite adicionar novas tarefas.



Refatorando para componentes filhos (ListaTarefas.razor)

Home.razor Pai

Envia Lista de tarefas

Usa o componente **ListaTarefas**

E contém a lógica para excluir uma tarefa

Remove da lista e atualiza a tela

envia a lista de
tarefas

Ao disparar o
evento usa o tipo
EventCallback e
notifica o Pai

ListaTarefas.razor Filho

Recebe a lista de tarefas

Contém a lógica para exibir as tarefas

E o botão para excluir uma Tarefa
chama **RemoverTarefa(id)**

Vamos extrair toda a lógica de exibição da lista de tarefas para um novo componente

Como o componente filho (**ListaTarefas**) pode notificar o componente pai (**Home**) que uma tarefa deve ser removida?

Resposta: Usando **EventCallback**

No Blazor, **EventCallback** é um tipo especial de **parâmetro** que permite a um componente **filho** notificar seu componente **pai** sobre um evento.



Componentes aninhados e comunicação Filho -> Pai

ListaTarefas.razor

```
@if (ListaDeTarefas == null)
    <Aviso Mensagem="Carregando tarefas..." />
}
...
@code{
    List<Tarefa> tarefas = new List<Tarefa>();
    ...
}
```



TarefaIndividual.razor

```
<td>
    .DataCriacao.ToShortDateString()</td>
    <td>@TarefaItem.Descricao</td>
    <td><input type="checkbox"
    @bind="TarefaItem.Concluida" /></td>
    ..
</tr>
...
```

Renderiza cada linha da tabela

E possui o botão De Remover a tarefa

Como o componente filho (**TarefaIndividual**) pode notificar o componente pai (**ListaTarefas**) que uma tarefa deve ser removida?

Resposta: Usando **EventCallback**



Componentes aninhados e comunicação Filho -> Pai

Home.razor

Pai

```
<ListaTarefas ListaDeTarefas="@tarefas" OnRemoverTarefa="RemoveTarefa" />
```

Lógica de exclusão - RemoveTarefa

A lógica de exclusão da tarefa esta neste componente

ListaTarefas.razor

Filho

```
<TarefaIndividual TarefaItem="item" OnRemoverTarefa="HandleRemoveTarefa" />
```

Cria a tabela HTML

[Parameter]

```
public EventCallback<Guid> OnRemoverTarefa { get; set; }
```

O **ListaTarefas** recebe o aviso do seu "filho" (**TarefaIndividual**) através do **EventCallback** dele. No entanto, o **ListaTarefas** não sabe como remover uma tarefa. A lógica de remoção (**RemoveTarefa**) está no **Home.razor**. Assim, o **ListaTarefas** precisa repassar essa informação para o seu próprio "pai" (**Home.razor**). Ele faz isso usando seu próprio **EventCallback** que foi passado pelo **Home.razor**.

TarefaIndividual.razor

Neto

Renderiza cada linha da tabela

[Parameter]

```
public EventCallback<Guid> OnRemoverTarefa { get; set; }
```

O **botão de remover** está neste componente. Quando o usuário clica nele, o **TarefaIndividual** precisa avisar seu "pai" (**ListaTarefas**) que uma tarefa deve ser removida. Ele faz isso usando seu próprio **EventCallback** (**OnRemoverTarefa**).



Componentes aninhados e comunicação Filho -> Pai

Hieraquia de Componentes

Home.razor (Pai de Nível Superior / Página)

| --- <**Aviso**> (Filho de Home)

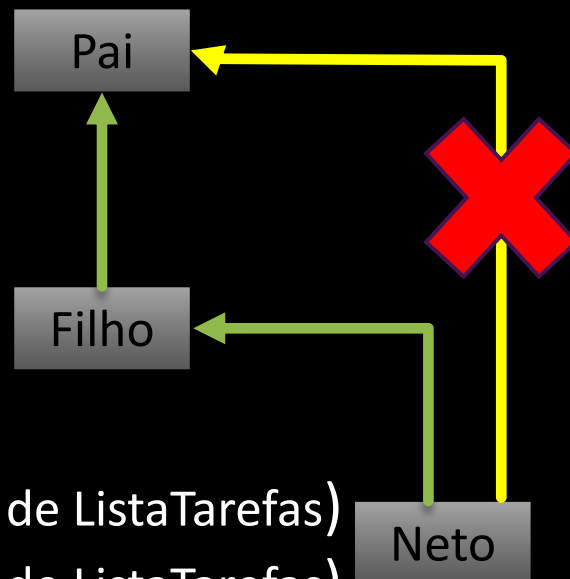
| | --- <**ListaTarefas**> (Filho Home)

| | | --- <**TarefaIndividual**> (Filho de ListaTarefas)

| | | --- <**TarefaIndividual**> (Filho de ListaTarefas)

| | | --- ... (um para cada tarefa)

| ----- (Formulário de Adicionar Tarefa)



O Blazor não permite que um "**neto**" se comunique diretamente com o "**avô**". O **TarefaIndividual** não tem conhecimento direto do **Home.razor** nem do método **RemoveTarefa**. Ele só conhece a tarefa que recebeu e o **EventCallback** que seu pai (**ListaTarefas**) lhe deu.

@ Hierarquia de componentes - justificativa

Busca a lista de tarefas via HTTP.
Mantém a lista de tarefas e a nova tarefa a ser adicionada.
Controla a adição e remoção de tarefas.
Renderiza a interface principal e delega a renderização da lista para o componente **ListaTarefas**.

Home.razor

Recebe **a lista de tarefas** via parâmetro.
Recebe um **EventCallback** para remoção de tarefas.
Pode delegar a renderização de cada tarefa para o componente **TarefaIndividual**.

ListaTarefas.razor

Renderiza os detalhes da tarefa.
Permite marcar a tarefa como concluída.
Permite remover a tarefa, notificando o **componente pai (via EventCallback)**

TarefaIndividual.razor

Podemos reutilizar o componente **TarefaIndividual** em diferentes contextos para mostrar uma tarefa sem repetir o código

Se no futuro quiser mudar a forma como uma tarefa é exibida (*ex: adicionar um botão de editar, mudar o layout*), basta alterar o componente **TarefaIndividual** e a mudança será refletida em todos os lugares onde ele é usado.

Alterações específicas na exibição ou comportamento de **um item de tarefa** ficam isoladas no componente **TarefaIndividual**, evitando que mudanças afetem o componente de lista inteira

Testar e depurar fica mais simples, pois você pode focar em um componente menor e mais específico.

@ A propagação do modo de renderização

Home.razor

Pai

```
@rendermode InteractiveServer  
<ListaTarefas ListaDeTarefas..../>
```

Definimos a diretiva **@rendermode InteractiveServer** neste componente. Isso instrui o Blazor a estabelecer uma conexão **SignalR** para este componente e a lidar com seus eventos de UI (como cliques de botão) no servidor.

ListaTarefas.razor

Filho

```
<TarefaIndividual Tarefatem... />
```

Este componente é renderizado dentro de **Home.razor**. Como ele não especifica um modo de renderização próprio, ele automaticamente herda o modo de seu pai sendo renderizado no modo **InteractiveServer**.

TarefaIndividual.razor

Neto

```
public EventCallback<Guid>.....
```

Este componente é renderizado dentro de **ListaTarefas.razor**. Seguindo a mesma lógica, ele herda o modo de renderização de **ListaTarefas**, que é **InteractiveServer**.

Você só precisa definir o modo de renderização **no componente de nível mais alto** que precisa de interatividade. Todos os **seus descendentes se tornarão interativos automaticamente**.